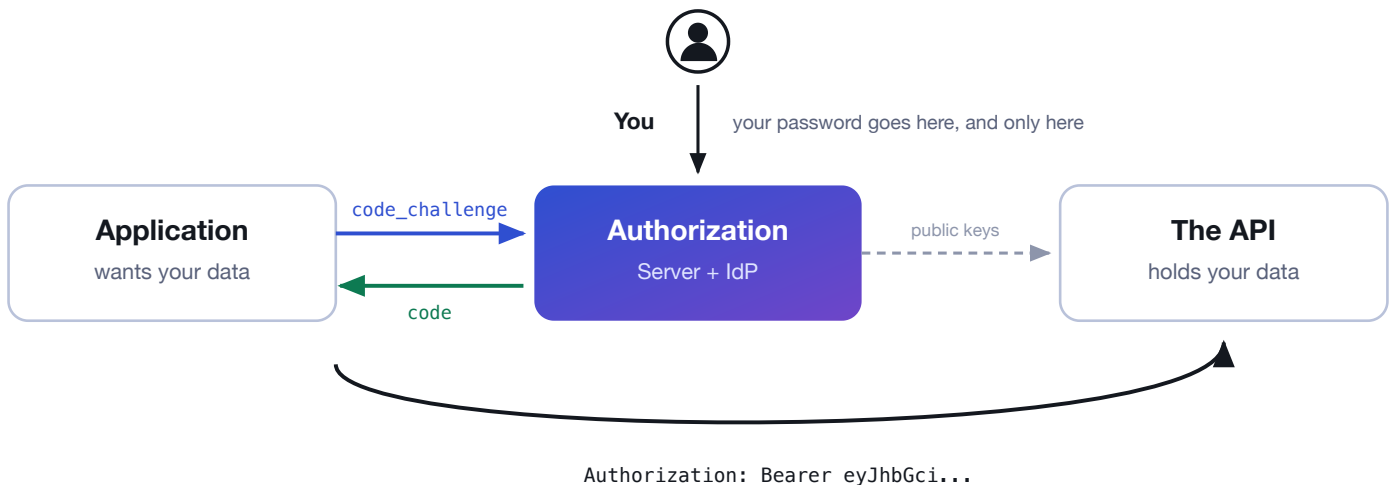


OAuth 2.1 and PKCE

How delegated authorization actually works — from the problem it solves to the cryptography that makes it safe. Written for developers meeting OAuth for the first time, and for anyone who has used it for years without being sure why.



The application never learns your password. The API never asks who you are.

Covers

Authorization Code flow · PKCE · tokens & scopes · refresh rotation · what OAuth 2.1 removed · the attacks each rule prevents

Companion

A runnable simulator ships alongside this guide. Every diagram here corresponds to a lab you can execute.

What is in this guide

Read it in order the first time. Each chapter assumes only the ones before it.

01	The problem OAuth solves	Why handing over your password is the wrong answer	3
02	The four roles	Resource owner, client, authorization server, resource server	4
03	Front channel and back channel	The one distinction the whole protocol is built on	5
04	The Authorization Code flow	All ten steps, drawn and then narrated	6
05	The attack PKCE prevents	What a stolen authorization code used to be worth	10
06	PKCE, precisely	Three lines of arithmetic, and why they are enough	11
07	Anatomy of a token	Inside a JWT; access vs ID vs refresh tokens	13
08	Scopes and consent	How authority is narrowed, three times over	15
09	Refresh tokens and rotation	Long sessions from short-lived tokens	16
10	What OAuth 2.1 changed	Mostly a list of deletions, and that is the point	17
11	How an API validates a token	Four checks, in order, on every request	18
12	The mistakes that matter	A checklist drawn from real incidents	19
13	Run it yourself	Every concept here, as a lab in the simulator	20
–	Glossary	Every term in one place	21
–	The specifications	Which RFC says what, and which to read first	22

How to read this with the simulator

Two windows, side by side

Keep `localhost:9020` (the labs) next to `localhost:9000/trace` (the authorization server's live decisions). Reading a rule here and then watching it fire there is the fastest way to make any of this stick.

Break things deliberately

The simulator has a **policy page** where each OAuth 2.1 requirement can be switched off at runtime. Whenever this guide says “this rule prevents X”, you can turn the rule off and watch X happen.

A note on terminology. OAuth is about **authorization** (what an app may do), not **authentication** (who you are). The two get muddled constantly because in practice they arrive together. Chapter 7 separates them: authentication is what OpenID Connect adds on top.

The problem OAuth solves

An application needs data that belongs to you, held by somebody else. There is an obvious way to arrange that, and it is a catastrophe.

Suppose a budgeting app wants to read your bank transactions. The naive design is to ask you for your banking username and password and log in as you. Every single thing wrong with that idea turns out to be something OAuth fixes.

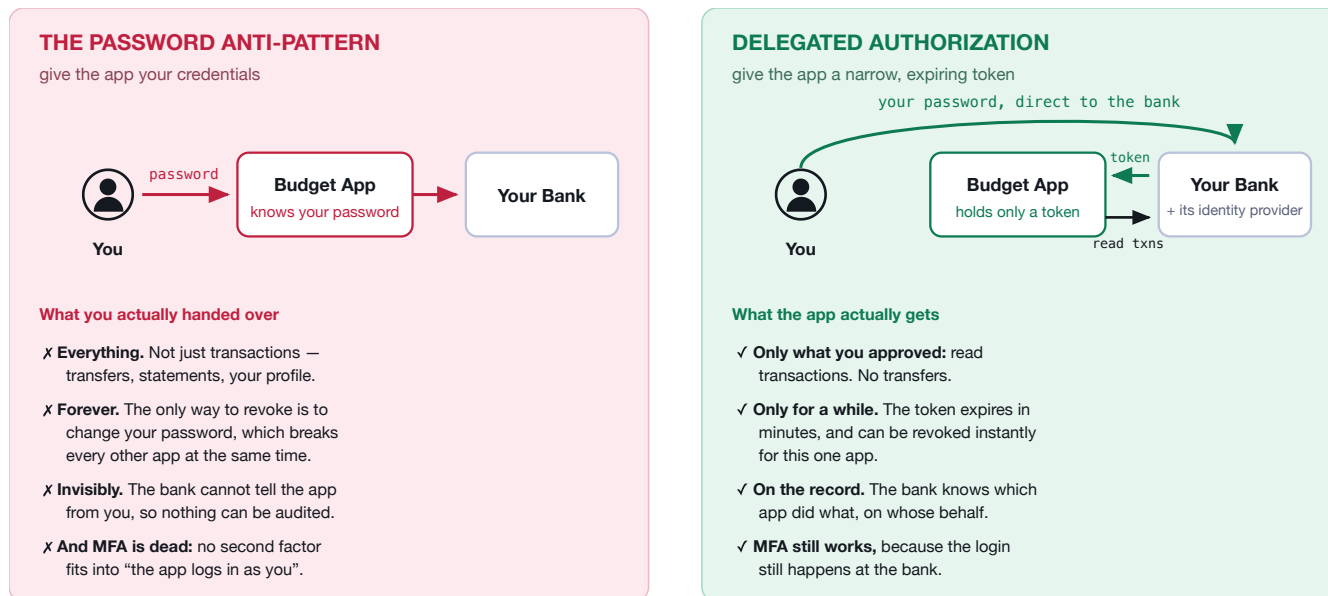


Figure 1.1 — The same goal, two arrangements. On the right, the password travels only between you and the party that issued it; the application is handed a credential that is narrow by construction.

The one-sentence definition

OAuth 2.1 is a protocol for letting an application act on your behalf, in a limited way, without ever seeing your password. You authenticate to the party that already knows you; that party issues the application a scoped, expiring, revocable token.

The rest of this guide is about doing that *safely* when the exchange must pass through a web browser — the single constraint, explored in Chapter 3, that makes OAuth look harder than the sentence above.

Three things OAuth is not

Not authentication

An access token says *what may be done*, not *who is here*. Treating one as proof of identity is a vulnerability class of its own; identity is a separate token (Chapter 7).

Not encryption

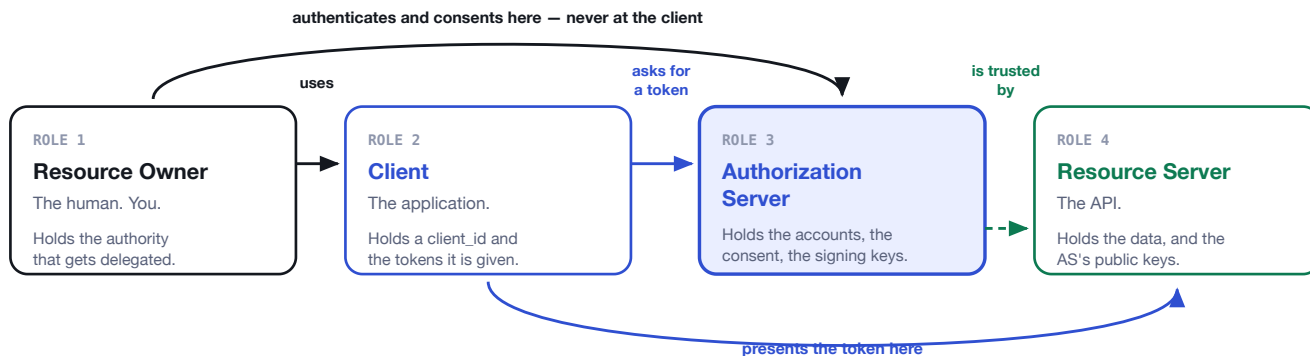
A bearer token is a password-equivalent in plain text: anyone who copies it can use it. OAuth assumes **TLS everywhere**.

Not a permission system

Scopes are coarse labels on an app's authority. Whether *this user* may touch *this record* stays your API's decision, on every request.

The four roles

OAuth defines four participants. Almost every confusing sentence in the specifications becomes clear once you know which of the four is being talked about.



The asymmetry that makes it work:

the **client** never learns who you are, and the **API** never learns how you proved it. Each knows only what it needs.

Figure 2.1 — The four roles and the relationships between them. Roles 3 and 4 are often the same company but never the same code path: issuing tokens and accepting them are different jobs.

The same four roles, three ways

ROLE	BANK EXAMPLE	“SIGN IN WITH GOOGLE”	IN THE SIMULATOR
Resource Owner	You, the account holder	You, the Google user	alice or bob
Client	The budgeting app	The site you are signing in to	:9020
Authorization Server	The bank's login service	Google's OAuth service	:9000
Resource Server	The bank's accounts API	Gmail / Drive / Calendar APIs	:9010

Two kinds of client, and why it matters so much

This distinction decides how a client may authenticate, and it is why PKCE exists.

Confidential client HAS A SECRET

Code that runs on a server you control. It can keep a `client_secret` because nobody can read its memory or disk.

Examples: a server-rendered web app, a backend service.
Proves identity by: sending its secret to the token endpoint.

Public client CANNOT KEEP A SECRET

Code that runs on someone else's device. Anyone can read the JavaScript bundle or decompile the binary, so any embedded secret is published to the world.

Examples: a single-page app, a mobile app, a CLI. **Proves identity by:** PKCE, and nothing else.

The mistake to avoid. A single-page app is a **public** client however private your bundle feels: a secret shipped to a browser or an app store is not a secret. OAuth 2.1's answer is to stop depending on one — PKCE is mandatory for every client, so correctness no longer rests on this classification.

Front channel and back channel

If you remember one idea from this guide, make it this one. Every rule in OAuth 2.1 — including all of PKCE — follows from the difference between these two ways of moving data.

The client and the authorization server need to talk. They have two completely different means of doing so, with completely different security properties.

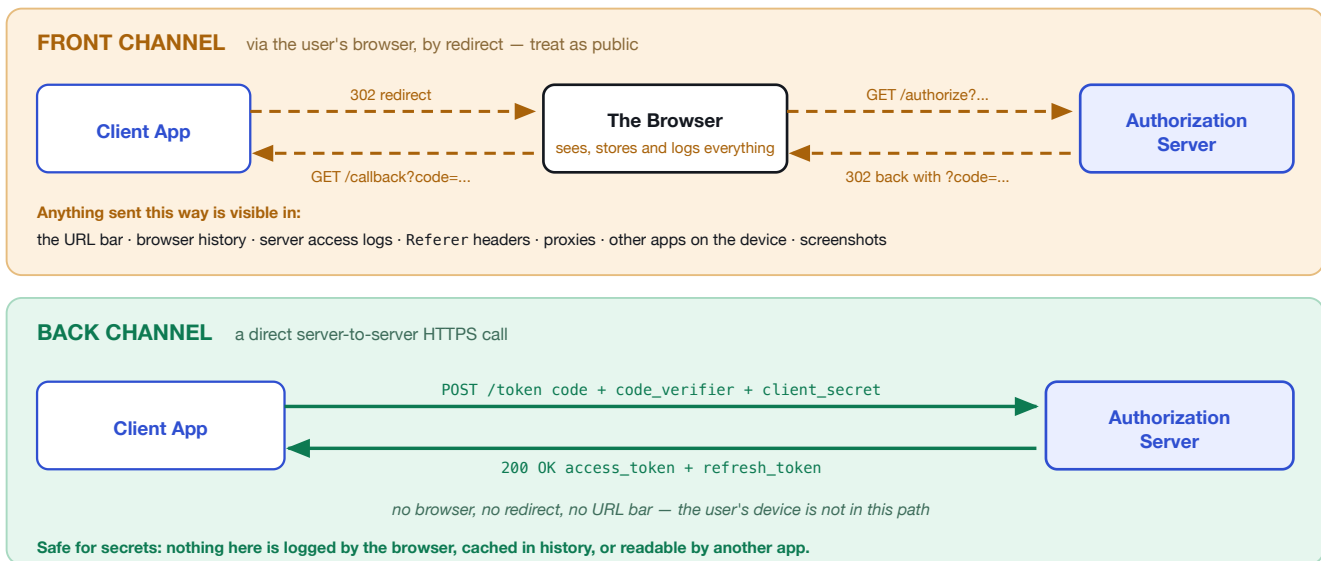


Figure 3.1 — The same two parties, two channels. Dashed arrows are browser redirects; solid arrows are direct HTTPS calls.

The rule that follows

May cross the front channel

- `client_id` — public by definition
- `redirect_uri`, `scope`, `state`
- `code_challenge` — a hash, not a secret
- the `code` — useless on its own (*this is the whole trick*)

Must never cross it

- `client_secret`
- `code_verifier`
- `access_token`, `refresh_token`, `id_token`
- the user's password, obviously

This is why the implicit grant was removed. It returned an *access token* on the front channel, in the URL fragment — landing it in browser history and in any Referrer header the page emitted. Chapter 10 has the full list of what OAuth 2.1 deleted and why.

“Front channel” does not mean unencrypted

Both channels run over TLS in production. The front channel is dangerous not because it is exposed in transit, but because of where the data **comes to rest**: browsers write URLs into history, servers into access logs, pages leak them through `Referer`. TLS does nothing about any of that. OAuth 2.1 still requires `https` everywhere, with two carve-outs — `http://localhost` for development, and private-use schemes like `com.example.app:/callback` for native apps. That is why the simulator may run on plain HTTP, and why nothing else should.

The Authorization Code flow

All of OAuth 2.1 in one picture; the next two pages walk it step by step.

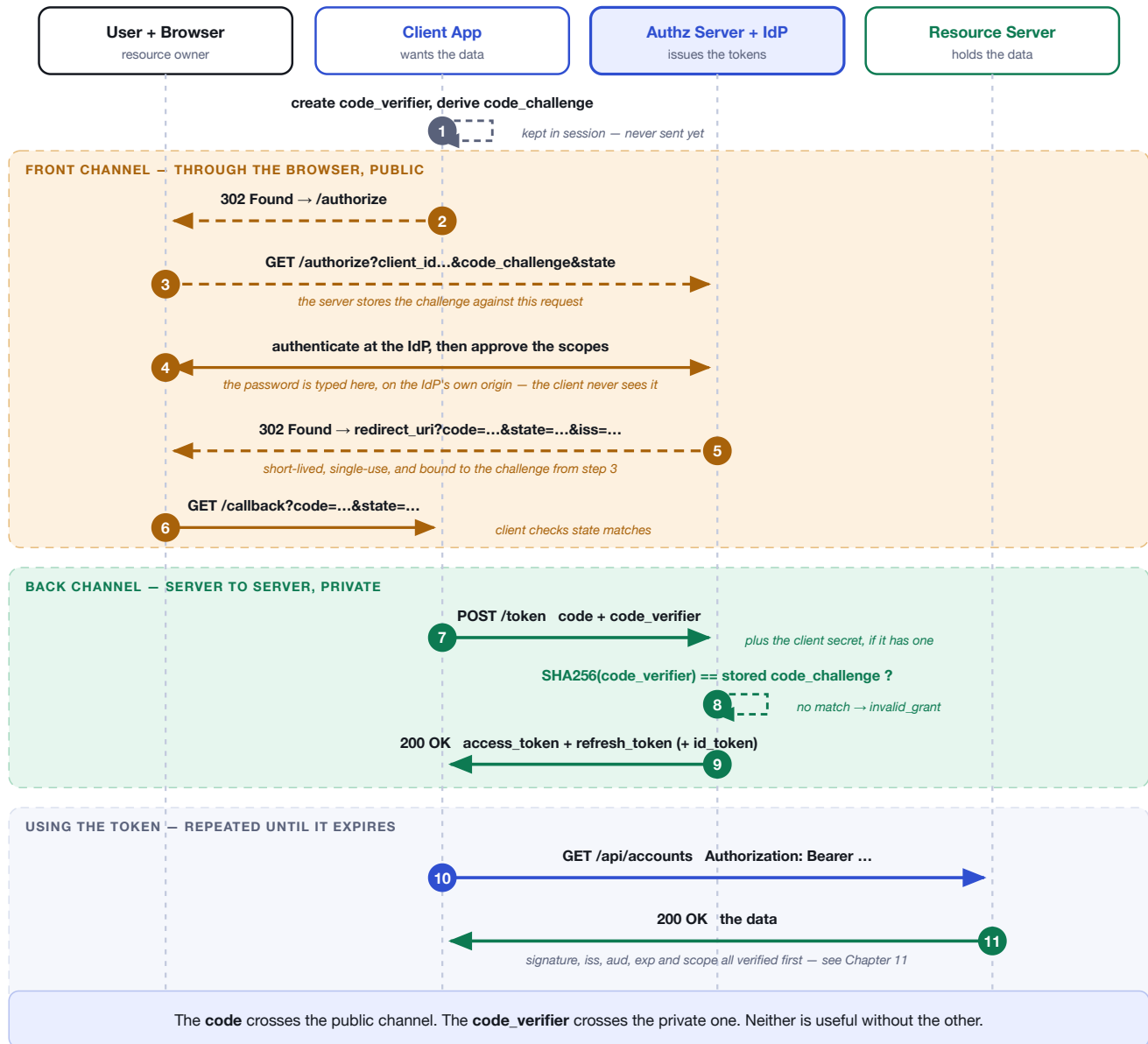


Figure 4.1 — Dashed arrows are browser redirects, solid arrows are direct HTTP calls, looped arrows are local computation. Steps 10–11 repeat until the token expires.

Three things to notice

The user is a participant

Steps 2–6 happen in a browser because a human has to be asked. That is not overhead — it is the consent.

The code is a claim ticket

It is not a token and grants nothing: a single-use reference, valid about a minute, redeemable only by whoever started the flow.

Two channels, one flow

The split at step 7 is the design: the risky public hop carries something worthless, the valuable exchange happens out of reach.

Step by step: the front channel

The same flow again, this time as real HTTP. Every value below was taken from an actual run of the simulator.

1 · The client invents a secret it will not send

Before anything touches the network, the client generates a high-entropy random string and hashes it. The random string is the **code_verifier**; the hash is the **code_challenge**.

```
# 32 random bytes, base64url-encoded => exactly 43 characters
code_verifier = 3TbF_QUTB0iHthQd0rRr_6GcCTQKN_nPqc9j3P0FCBg

# BASE64URL(SHA256(ASCII(code_verifier)))
code_challenge = kPX0H7VuZIrt1ZXIk7n0C2HkA6r0YNI3lDys2XF1Crk
```

The verifier is stored in the client's own session. It must survive until step 7 and must never appear in a URL, a cookie readable by script, or a log line.

2–3 · Redirect the user to the authorization endpoint

The client does not call `/authorize` itself. It sends a `302` so that the *browser* goes there, carrying the parameters in the query string.

```
GET /authorize?response_type=code
    &client_id=demo-web-app
    &redirect_uri=http%3A%2F%2Flocalhost%3A9020%2Fcallback
    &scope=openid+profile+accounts%3Aread
    &state=cclt66TixPmn7BXxpsyNEA
    &code_challenge=kPX0H7VuZIrt1ZXIk7n0C2HkA6r0YNI3lDys2XF1Crk
    &code_challenge_method=S256

Host: localhost:9000
```

PARAMETER	WHAT IT IS FOR
<code>response_type</code>	Always <code>code</code> in OAuth 2.1. The other values are gone.
<code>client_id</code>	Which application is asking. Public, not a credential.
<code>redirect_uri</code>	Where to send the code. Compared by exact string match against registration.
<code>scope</code>	What the application wants to be allowed to do. Ask for the least you need.
<code>state</code>	Random, per-request. Echoed back so the client can recognise its own callback.
<code>code_challenge</code>	The hash from step 1. The server stores it against this request.
<code>code_challenge_method</code>	<code>S256</code> . Send it explicitly — omitting it means <code>plain</code> , which protects nothing.

Step by step: the exchange

The user takes over, and hands back a code that is worth nothing by itself. Turning it into a token takes a direct call, and a proof.

4–6 • The user authenticates, consents, and comes back with a code

Everything in this step happens on the authorization server's own origin. The client is not running, cannot see the password, and cannot influence the consent screen. When it is done, the server redirects the browser back:

```
HTTP/1.1 302 Found
Location: http://localhost:9020/callback?code=bnBKJfGg0H4Mm_fzI4bE622-5t0w7U8kbKIHpubbQ4o
&state=cclt66TixPmn7BXxpsyNEA
&iss=http://localhost:9000
```

Two checks the client must make here. First, state has to equal the value it generated in step 2 — otherwise this callback was not started by this browser session, and the code must be dropped. Second, iss must name the authorization server the flow began with (RFC 9207), which prevents a code from one server being replayed at another.

7 • Redeem the code, and prove you started the flow

This is a server-to-server `POST`. The browser is not involved and never sees any of it.

```
POST /token HTTP/1.1
Host: localhost:9000
Content-Type: application/x-www-form-urlencoded
Authorization: Basic ZGVtbY13ZWItYXBwOndYi1hcHAtc3VwZXItc2VjcmV0  <- only if confidential

grant_type=authorization_code
&code=bnBKJfGg0H4Mm_fzI4bE622-5t0w7U8kbKIHpubbQ4o
&redirect_uri=http://localhost:9020/callback
&code_verifier=3TbF_QUTB0iHthQd0rRr_6GcCTQKN_nPqc9j3P0FCBg
```

8 • What the authorization server checks, in order

1. Does this `code` exist, and has it been used before? A replay means someone has a copy; every token it produced is revoked.
2. Has it expired? Codes live about a minute, because they cross the browser.
3. Was it issued to *this* `client_id`? A code belongs to one client.
4. Does `redirect_uri` match the one from step 3? Both halves of the flow must agree.
5. Does `SHA256(code_verifier)` equal the stored `code_challenge`?
6. If the client is confidential, did it authenticate correctly?

Any failure returns `400 invalid_grant` and no token — and check 5 is the only one an attacker holding a stolen code cannot satisfy.

Step by step: the tokens

What comes back, how it is used, and what the failures mean.

9 · The tokens come back

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: no-store

{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwOi8v...",
  "token_type": "Bearer",
  "expires_in": 300,
  "scope": "openid profile accounts:read",
  "refresh_token": "hz9ZLvVJ2akqMFGb7pQx...",
  "id_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwOi8v..."
}
```

`Cache-Control: no-store` is required, not decorative: these are credentials. Note too that `scope` comes back in the response — the user may have granted **less** than was asked for, and a correct client reads what it received.

10–11 · Call the API

```
GET /api/accounts HTTP/1.1
Host: localhost:9010
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...
```

The token goes in the `Authorization` header — never in a query string, where it lands in every access log on the path. OAuth 2.1 removes the query-string option OAuth 2.0 allowed.

When it goes wrong: decoding the error

ERROR	USUAL CAUSE
invalid_grant	The code was used already, expired, issued to another client, or the <code>code_verifier</code> did not match. Deliberately vague: the server will not tell an attacker which check failed.
invalid_client	Wrong <code>client_id</code> or secret, or a confidential client that sent no credentials at all.
invalid_request	A required parameter is missing or duplicated. Commonly a forgotten <code>code_challenge_method</code> .
unauthorized_client invalid_scope	The client is not registered for this grant type, or asked for a scope it is not allowed (or that does not exist).
access_denied	The user pressed Deny. Not a bug; handle it gracefully.

That is the entire flow. Eleven steps, two channels, one hash comparison. Every remaining chapter is either a detail of what is inside the tokens, or a rule protecting one of these steps.

The attack PKCE prevents

The authorization code has to cross the browser, and the browser is not a private place. So assume the code leaks — and ask what it is worth.

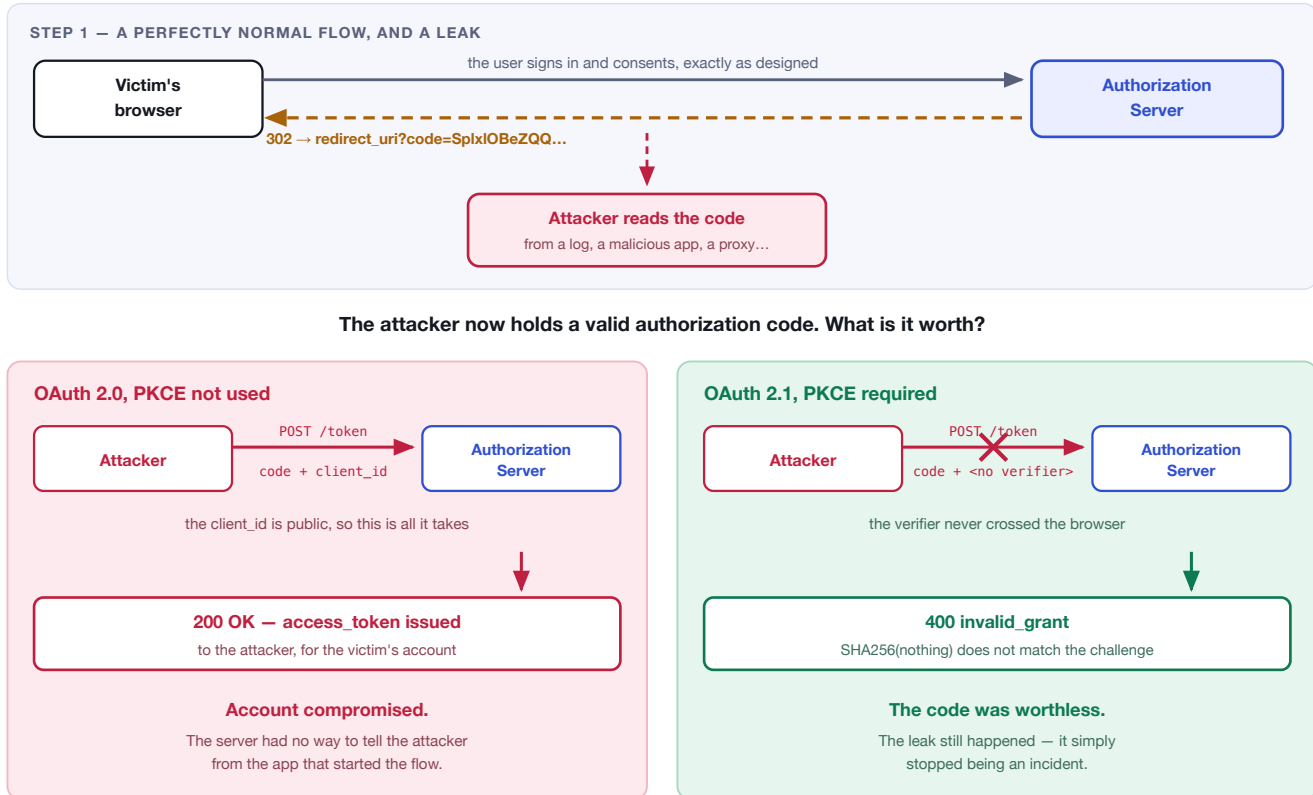


Figure 5.1 — Identical theft, two outcomes. PKCE does not prevent the code from leaking; it removes the value of having it.

How codes actually leak

None of these are exotic. Every one has produced real CVEs:

- **A malicious app on the same device** registering the same custom URI scheme (`myapp://callback`) — the OS may hand the redirect to either app.
- **Server access logs.** The callback URL, complete with code, is written to disk by every proxy and web server in the path.
- **The `Referer` header** leaking the callback URL to any third-party script or image on the landing page.
- **An open redirect** on the client's own domain, used to bounce the code to the attacker's server.
- **Browser history** on a shared or managed machine.
- **A TLS-terminating proxy** on a corporate or hostile network.

Why OAuth 2.1 requires PKCE even for confidential clients. A server-side app with a secret is already protected against most of this — the attacker would need the secret too. But PKCE also protects against a code being *injected* into an honest client's session, costs nothing, and removes any need to reason about which category a client falls into. Universal rules fail safe; conditional ones get misapplied.

PKCE, precisely

Proof Key for Code Exchange (RFC 7636). One random string, one hash, one comparison — and the authorization code stops being worth stealing.

The name is unhelpful; the idea is simple. Before starting the flow the client invents a secret, sends only a *fingerprint* of it through the browser, and later proves it holds the original. Only the app that began the flow can finish it.

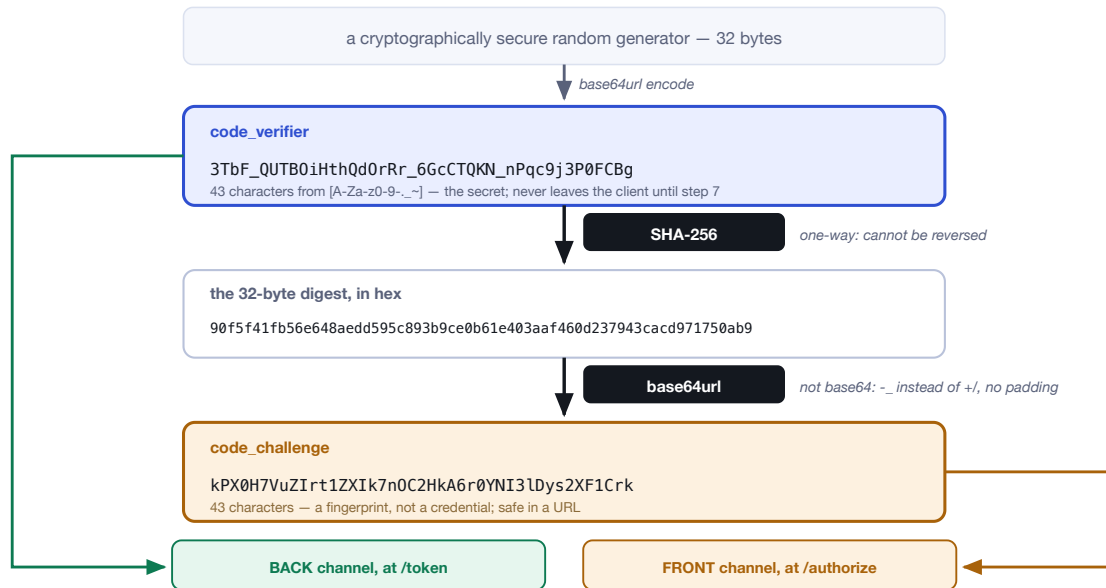


Figure 6.1 — Deriving the challenge from the verifier, with real values. The two results travel on different channels, and only one of them is a secret.

Why a hash is enough

The attacker sees the challenge

It is right there in the authorization URL. To use the stolen code they would need a string that hashes to it — that is a SHA-256 preimage, which nobody can compute. The challenge is public *by design*.

The attacker cannot guess the verifier

It is 256 bits of randomness. There is no dictionary, no pattern and no structure to attack. This is why the specification insists on a *cryptographically secure* generator, not `Math.random()`.

Never use `code_challenge_method=plain`. With `plain`, the “challenge” is the verifier, sent in the clear through the browser — so anyone who sees the authorization request can redeem the code. It exists in RFC 7636 only for devices that genuinely could not compute SHA-256 in 2015. Worse, RFC 7636 says that an **omitted** `code_challenge_method` defaults to `plain`. Always send S256 explicitly.

The check, and how to write it

What the server does with the two halves, and the code that produces them.

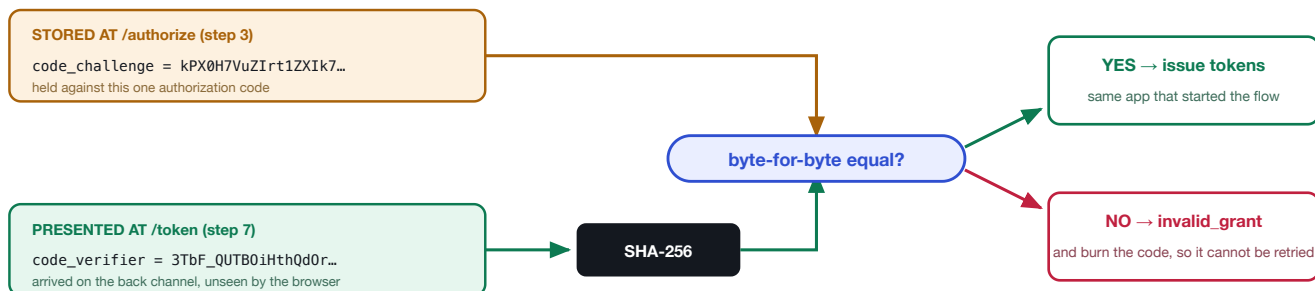


Figure 6.2 — The whole verification. Compare in **constant time**, so a rejection cannot be timed byte by byte, and treat a failure as a security event, not a retry.

Generating the pair

Browser (Web Crypto)

```

const b64url = (b) => btoa(
  String.fromCharCode(...b))
  .replace(/\+/g, '-')
  .replace(/\//g, '_')
  .replace(/=+$/, '');

const verifier = b64url(
  crypto.getRandomValues(new
    Uint8Array(32)));

const hash = await crypto.subtle.digest(
  'SHA-256',
  new TextEncoder().encode(verifier));

const challenge = b64url(new
  Uint8Array(hash));
  
```

Node.js

```

const crypto = require('node:crypto');

const verifier = crypto.randomBytes(32)
  .toString('base64url');

const challenge =
  crypto.createHash('sha256')
    .update(verifier).digest('base64url');
  
```

Python

```

import base64, hashlib, os

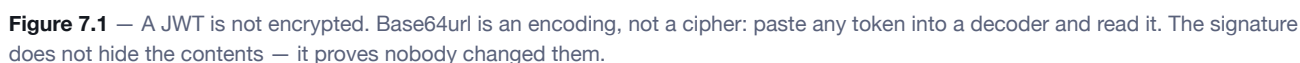
b64 = lambda b:
  base64.urlsafe_b64encode(b).rstrip(b'=')

verifier = b64(os.urandom(32))
challenge =
  b64(hashlib.sha256(verifier).digest())
  
```

In all three, the SHA-256 is taken over the **base64url text** of the verifier, never over the raw random bytes. Getting that backwards is the most common PKCE bug and the hardest to spot, because both values still look like valid base64url.

One pair per authorization request. A verifier is a nonce, not a configuration value: generate a fresh one every time and discard it the moment the code is exchanged. On the server side, compare with `timingSafeEqual` rather than `==`, and treat a mismatch as a security event — invalidate the code, return the generic `invalid_grant`, and never reveal which check failed. The common implementation mistakes are collected in Chapter 12.

eyJhbGciOiJSUzI1NiIsInR5cCI6ImlmF0k2p3dCI6ImtpZCI6InNpbS1FYXJIZKNO.
eyJpc3MiOiJodHRwOi8vbG9yYXRob3N0OjkwMDA1LlZjdWI0Ij11LEwMDE1LlZj.
0kNChhbG9y TLJLT0auscjcfnwn2VKKzJZPB0fzVrsqjNsesPiNaCJaZZaq50xl



What the segments say once decoded

Plenty of servers issue random strings instead. The API then calls `/introspect` to find out what a token means. Same protocol, different trade-off — see Chapter 11.

13

Three tokens, three jobs

A single token response can contain three completely different things. Confusing two of them is the most common OAuth bug in production code.

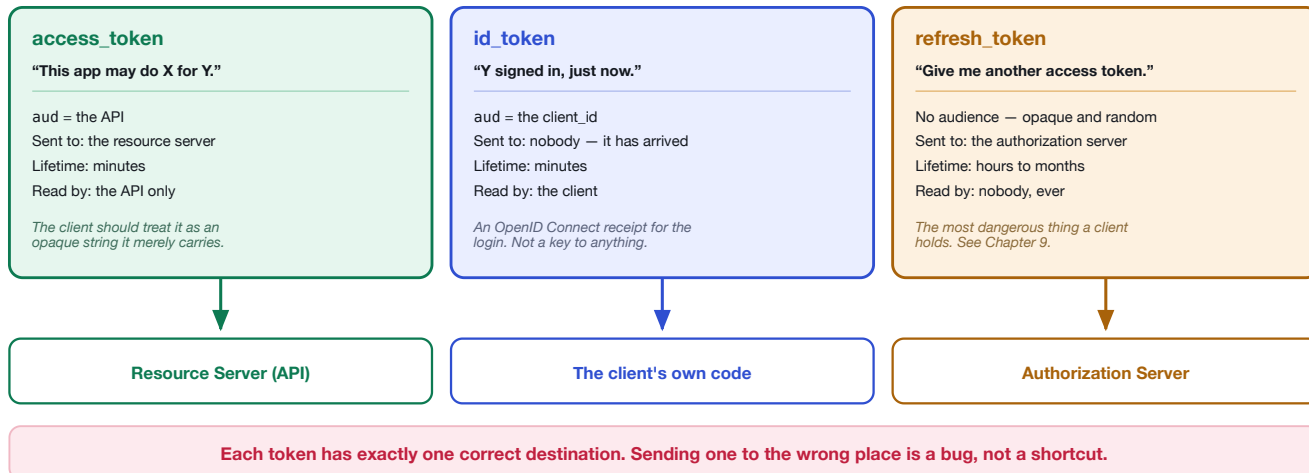


Figure 7.2 — Three tokens, three consumers. The aud claim is not documentation: it is the mechanism that enforces this separation.

The mistake, and why the API must refuse it

An ID token is a JWT from the same issuer, so it is tempting to send it as a Bearer token. It looks valid. It is not:

```
# The ID token's own claims say who it is for:
"aud": "demo-web-app"           # the CLIENT, not the API
"typ": "JWT"                    # not at+jwt
# ...and it carries no `scope` at all, so there is nothing to authorize.
GET /api/accounts Authorization: Bearer <id_token> -> 401 invalid_token
```

An API accepting an ID token is accepting a credential minted for somebody else, with no statement of what it may do — and in a federated setup that is exploitable.

The habit to build. When a call fails with 401, check which token you sent before you check anything else. “I sent the ID token” accounts for a remarkable share of lost afternoons. Lab 08 in the simulator does exactly this, twice.

Where OpenID Connect fits

OAuth alone never tells the client *who* the user is; it only hands over authority. OpenID Connect is a thin layer that adds exactly that. “Sign in with Google” is OpenID Connect, “let this app read your Drive” is plain OAuth, and most deployments do both at once in the one flow from Chapter 4:

- Ask for the `openid` scope, and the token response gains an `id_token`.
- Send a random `nonce`; it comes back inside the ID token, proving the token was minted for *this* login rather than replayed.
- A `/userinfo` endpoint returns profile claims, filtered by granted scope.

Scopes and consent

A scope is a label for a slice of authority. What an application asks for and what it gets are two different things, and it is narrowed three separate times.

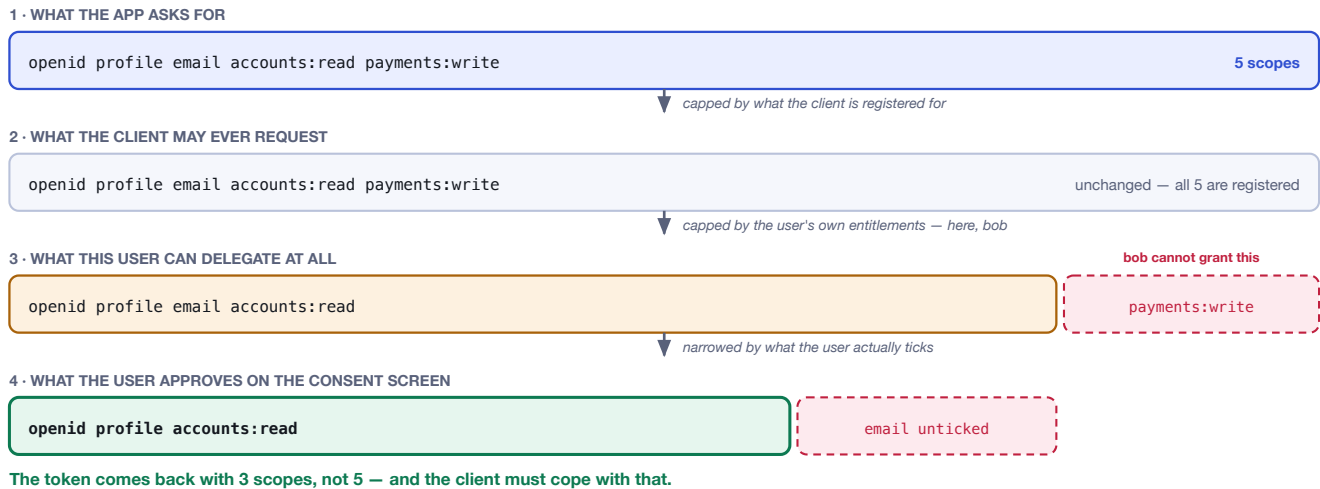


Figure 8.1 — Scope only ever narrows. Each stage can remove, never add, which is why the token response repeats the scope it actually granted.

Designing scopes people can consent to

Good

- `accounts:read`, `payments:write` — resource plus verb
- Separate read from write, always
- Few enough to fit on one screen
- Named for what the *user* loses if abused

Bad

- `admin`, `full_access` — unconsentable
- One scope per API endpoint — unreadable
- Scopes encoding record ids (`account:4410`)
- Internal role names leaked into the UI

Scope is not the same as permission

This distinction causes real vulnerabilities. A scope says *this application may perform this kind of operation on this user's behalf*. It says nothing about which records that user may touch.

```
# The token carries scope=accounts:read and sub=u-1001.
GET /api/accounts/ACC-7100          # <- an account belonging to u-1002

# A scope check alone would ALLOW this. The API must also check ownership.
```

The simulator's API takes the user from the token's `sub` and never from the request, which is the habit worth copying. An object reference supplied by the caller must always be authorized against the token's subject.

Refresh tokens and rotation

Access tokens expire in minutes. Users expect to stay signed in for weeks. The refresh token bridges that gap — and needs careful handling because of it.

An access token cannot be recalled once issued, so it is given a short life: a leak is a problem for five minutes, not five months. The refresh token mints replacements without troubling the user.

ONE TOKEN FAMILY, OVER TIME

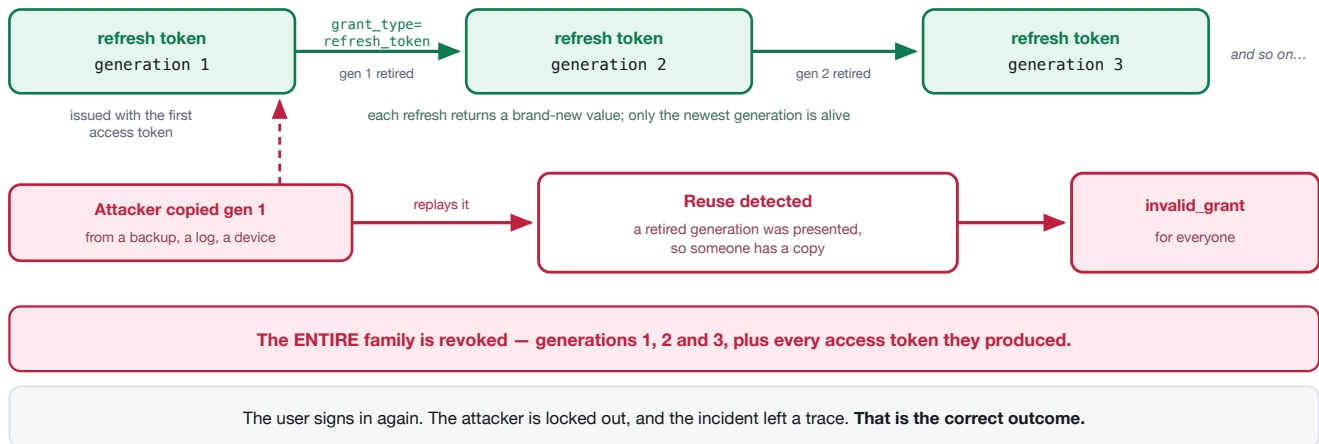


Figure 9.1 — Rotation with reuse detection (RFC 9700 §4.14). The server cannot tell thief from victim, so it trusts neither: a silent, permanent compromise becomes a visible, recoverable one.

What this buys you

- **A stolen refresh token stops working.** Whichever party refreshes second is refused, and the theft is detected rather than lasting for months.
- **The theft becomes observable.** A reuse event is a high-quality security signal, because correct clients never replay a retired token. This is why OAuth 2.1 requires rotation (or sender-constraining) for public clients.

How long should each one live?

CREDENTIAL	TYPICAL	WHY
authorization code	30–60 s	It crosses the browser and is redeemed immediately. Anything longer is pure exposure.
access token	5–15 min	Cannot be recalled, so its lifetime is the exposure window.
refresh token	hours–months	Long, but rotating and revocable. Shorter for public clients.
IdP session	hours	Decides whether a <i>new</i> authorization needs a fresh login.

The operational catch. If a client crashes after sending a refresh but before storing the new token, it will replay the old one and get the whole family revoked — logging the user out for no good reason. Persist the rotated token before you use the access token that came with it, and serialise refreshes so two tabs cannot race.

What OAuth 2.1 changed

OAuth 2.1 invents almost nothing. It deletes what proved dangerous and makes the best existing practice mandatory — and that is precisely why it is an improvement.

OAuth 2.0 is a framework of options. A decade of deployments showed which options were traps, and the security Best Current Practice (RFC 9700) documented them. OAuth 2.1 consolidates that guidance into the core, so that the default reading of the specification is now the safe one.

OAUTH 2.0 (2012)	OAUTH 2.1	WHY IT CHANGED
PKCE optional, recommended for mobile	Required for every authorization code request	Codes leak in ways nobody predicted, the defence costs almost nothing, and a universal rule cannot be misapplied.
Implicit grant (<code>response_type=token</code>)	Removed	It put an access token in the URL fragment: browser history, <code>Referer</code> headers, any script on the page — and the client could not be authenticated at all.
Password grant (<code>grant_type=password</code>)	Removed	The application handled the real password, so there was no consent, no MFA, no federation, and no way for a user to tell a real app from a phishing one.
<code>redirect_uri</code> matching left to the server	Exact string comparison	Wildcards and prefix rules let an attacker append a path and have codes delivered to themselves.
Bearer tokens allowed in query strings	Header only	Query strings are written to every access log, proxy log and browser history on the path.
Refresh tokens static and long-lived	Rotate, or bind to the client	A stolen static refresh token is permanent, undetectable access. Rotation makes the theft visible and recoverable.
Little guidance on mixing up servers	iss in the authorization response	Prevents a mix-up attack where a code from a malicious server is replayed at an honest one.

If you already know OAuth 2.0

You know OAuth 2.1. Add PKCE everywhere, stop using implicit and password, match redirect URIs exactly, and rotate refresh tokens. That is the entire migration.

On the specification's status

OAuth 2.1 is an IETF draft that consolidates published, stable RFCs. Nothing in it is speculative — every requirement already exists as a standard somewhere. Implementing it is not a bet on a draft.

The pattern worth noticing. Every deletion removes a *choice* a developer could get wrong. The most reliable security improvement is not a new mechanism but the removal of an unsafe option — which is the actual lesson of OAuth 2.1.

How an API validates a token

The resource server is the part most tutorials skip. On every single request it asks four questions, in this order.

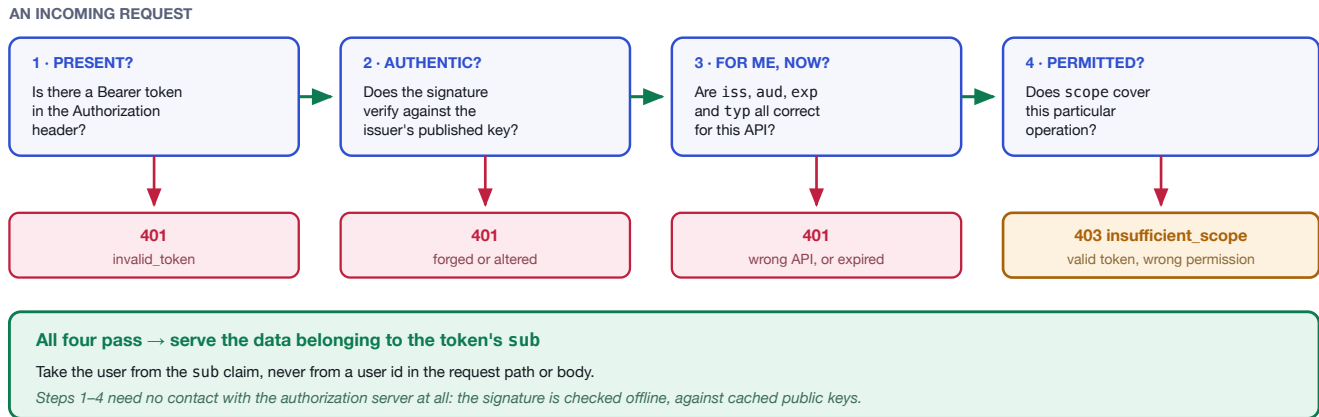


Figure 11.1 — The four gates. Note the status codes: **401** means “this token is not acceptable”; **403** means “the token is fine but does not authorize this”. Returning 401 for a scope failure sends clients into pointless refresh loops.

Local validation or introspection?

Verify the JWT locally

Fetch the issuer's public keys once from `/jwks.json`, cache them by `kid`, verify offline.

Fast and needs no network. **But** it cannot see a revocation: a revoked token stays acceptable until `exp`.

Ask the authorization server

Call `/introspect` (RFC 7662) per request. Works with opaque tokens too.

Always current, including revocation. **But** it adds a round trip to every call and makes the AS a hard dependency.

This trade-off is the real reason access tokens are short-lived: a five-minute window is the compromise between not calling the AS on every request and not honouring a revocation for too long. Lab 12 in the simulator revokes a token and then calls both kinds of endpoint, so you can watch one accept it and the other refuse.

Three ways to get validation wrong

- **Trusting the token's own `alg` header.** If you let the token choose its algorithm, an attacker sends `alg: none` (no signature at all) or swaps RS256 for HS256 and signs with your *public* key as the HMAC secret. Pin the expected algorithm in your code and reject everything else.
- **Skipping the `aud` check.** Then any token from that issuer works at your API, including one minted for a low-value service that has no business here. A valid signature only says who issued a token, not who it was for.
- **Caching keys without handling rotation.** Cache by `kid`, refetch when an unknown `kid` appears, and rate-limit that refetch. An issuer that rotates keys should not take your API down — nor should an unknown `kid` let an attacker trigger unbounded fetches.

The mistakes that matter

A checklist drawn from the failures that actually recur.

Writing a client

MISTAKE	CONSEQUENCE
Hashing the raw random bytes instead of the base64url verifier text	Every exchange fails with <code>invalid_grant</code> and nothing in the error says why.
Using base64 instead of base64url	<code>+</code> , <code>/</code> and <code>=</code> get percent-encoded in the URL, and the comparison fails.
Reusing one <code>code_verifier</code> for every request	A single leak compromises every future flow. It is a nonce: generate a fresh pair each time.
Keeping the verifier or tokens in <code>localStorage</code>	Readable by any script on the origin, so one XSS takes everything. Keep them server-side, or in memory.
Omitting <code>code_challenge_method</code>	RFC 7636 defaults it to <code>plain</code> : a strict server rejects you, a lenient one silently drops your protection.
Assuming the granted scope equals the requested scope	Features break confusingly when a user declines one checkbox. Read <code>scope</code> from the response.
Sending the <code>id_token</code> to the API	<code>401</code> , and an afternoon lost. It is a login receipt, not an API key.

Running a server or an API

MISTAKE	CONSEQUENCE
Accepting the token's own <code>alg</code> header	Opens <code>alg: none</code> and RS256→HS256 confusion. Pin the algorithm server-side.
Not validating <code>aud</code>	A token minted for any client of that issuer works at your API.
Prefix or wildcard <code>redirect_uri</code> matching	An attacker appends a path and has your users' codes delivered to them.
Letting an authorization code be redeemed twice	Whoever holds a copy can keep minting tokens. Single-use, and revoke on replay.
Returning <code>401</code> for a scope failure	Clients refresh in a loop trying to fix something a refresh cannot fix. Use <code>403</code> .
Taking the user id from the request instead of <code>sub</code>	A textbook IDOR sitting on top of otherwise correct OAuth.

A useful instinct. When OAuth surprises you, ask *which channel is this on, and who can see it?* Nearly every mistake above is a value that ended up on the wrong channel, or a check skipped because the happy path worked without it.

Run it yourself

Reading about a protocol gets you halfway. The simulator that ships with this guide runs a real OAuth 2.1 deployment on your machine, and lets you break it on purpose.

Start it

```
cd oauth-simulator
make up          # Docker
make local       # or plain Node, no Docker
```

Then open **localhost:9020** and sign in as `alice` / `wonderland`. Stop with `make down`.

Or drive it from a terminal

```
make demo        # the whole flow in curl
make demo-attack # a stolen code, refused
make test        # every scenario at once
```

Every parameter is printed, so you can follow the protocol without a browser.

What is running

SERVICE	URL	ROLE IN THIS GUIDE
Client app + labs	:9020	The client of Chapter 2. Holds the labs and a flow inspector showing every request it made.
IdP + authorization server	:9000	Chapter 2's authorization server . Login, consent and tokens, plus a live trace and the policy switches.
Resource server	:9010	The API of Chapter 11, running all four gates on every request.

Each chapter, as a lab

LAB	WHAT IT DOES	CH.	LAB	WHAT IT DOES	CH.
01	Authorization Code + PKCE, end to end	3–4	08	Sends the ID token, and a wrong-audience token	7
02	Hands your code to an attacker; PKCE blocks it	5–6	09	Tampers with <code>redirect_uri</code>	10
03	Turns PKCE off; the same theft succeeds	5–6	10	<code>state</code> , CSRF and the <code>iss</code> defence	4
04	Exchanges with the wrong <code>code_verifier</code>	6	11	<code>client_credentials</code> : no user, no consent	2
05	A public client with no secret at all	2	12	Revokes a token; local validation accepts it	11
06	Rotation, then replays a retired refresh token	9	13	Resurrects implicit and password grants	10
07	Scope narrowing and <code>403</code>	8			

The page worth opening first. `localhost:9000/policy` lets you switch off any OAuth 2.1 requirement at runtime. Run Lab 02 with PKCE on, turn `require_pkce` off, run Lab 03, and watch the same stolen code become a working access token. Nothing in this guide teaches the point faster.

Glossary

Every term used in this guide, in one place.

access token

A credential saying what an application may do, on whose behalf, at which API, until when. Usually a signed JWT. Sent to the resource server.

authorization code

A short-lived, single-use reference handed to the client through the browser, exchanged at the token endpoint for tokens. Worthless on its own.

authorization server (AS)

The service that authenticates the user, collects consent, and issues tokens. Publishes its metadata and public keys.

aud — the “audience” claim: which recipient a token is for. A resource server must reject tokens naming anyone else.

back channel

A direct server-to-server HTTPS call, invisible to the browser. Safe for secrets. See Chapter 3.

bearer token

A token usable by whoever holds it, with no further proof required — like a password. Hence short lifetimes.

client

The application requesting access. **Confidential** if it can keep a secret (server-side), **public** if it cannot (SPA, mobile, CLI).

code challenge / code verifier

The PKCE pair. The verifier is a random secret held by the client; the challenge is its SHA-256 hash, safe to send through the browser.

front channel

Communication via browser redirects. Visible in URL bars, history, logs and `Referer` headers. Never carries secrets.

grant type

How a client obtains a token. OAuth 2.1 keeps `authorization_code`, `refresh_token` and `client_credentials`; `implicit` and `password` are gone.

identity provider (IdP)

The service holding user accounts and performing authentication. Usually packaged with the authorization server, but a distinct job.

ID token

An OpenID Connect JWT stating who signed in and when, addressed to the client. Not an API credential.

introspection — RFC 7662: an endpoint where a resource server can ask whether a token is still active. Sees revocation immediately.

iss — the “issuer” claim: which authorization server minted a token. Checked on every validation.

JWKS — JSON Web Key Set: the public keys an authorization server publishes so anyone can verify its signatures offline.

JWT

JSON Web Token: header, payload and signature, base64url-encoded and dot-separated. Signed, **not** encrypted.

kid — key id: names which published key signed a token, so keys can be rotated without downtime.

nonce

A random value echoed inside the ID token, proving it was minted for this login.

PKCE

Proof Key for Code Exchange (RFC 7636). Binds an authorization code to the application that requested it. Mandatory in OAuth 2.1.

refresh token

Exchanged for new access tokens without user interaction, and rotated on every use.

resource server

The API holding the data. Validates tokens and enforces scope.

scope

A label for a slice of authority (`accounts:read`): requested, capped by registration and entitlement, then approved by the user.

state

A random per-request value echoed back on the callback, so a client can recognise a flow it started.

The specifications

Which document says what, and a sane order to read them in.

Read these, in this order

DOCUMENT	WHAT IT GIVES YOU
RFC 9700 OAuth 2.0 Security BCP	Start here. The accumulated lessons of a decade of deployments, with the attacks explained. Almost everything OAuth 2.1 requires is justified in this document.
OAuth 2.1 draft-ietf-oauth-v2-1	The consolidated core: one specification with the dangerous options already removed. Read it after 9700 and it will feel inevitable.
RFC 7636 PKCE	Short and readable. The definition of <code>code_verifier</code> , <code>code_challenge</code> and <code>S256</code> , and the attack that motivated them.
RFC 6749 OAuth 2.0 core	The original framework. Still the normative text for most parameter definitions — but read it knowing which parts 2.1 deletes.
RFC 6750 Bearer token usage	How to send a token and what each error code means. This is where <code>401</code> vs <code>403 insufficient_scope</code> is defined.

The ones you will need eventually

DOCUMENT	WHAT IT COVERS
RFC 8414	Authorization server metadata: <code>/.well-known/oauth-authorization-server</code> , so clients discover endpoints instead of hard-coding them.
RFC 9068	JWT access tokens: the <code>at+jwt</code> type and the claims an API should expect.
RFC 7662 / 7009	Token introspection and revocation.
RFC 8252	OAuth for native apps: system browsers, custom URI schemes, and why an embedded web view is the wrong answer.
RFC 8707 / 9207	Resource indicators (audience selection) and the <code>iss</code> authorization-response parameter.
RFC 9449	DPoP: sender-constrained tokens, so a stolen token is useless without the client's key. The direction bearer tokens are heading.
OpenID Connect Core	Authentication on top of OAuth: ID tokens, <code>nonce</code> , <code>/userinfo</code> and the <code>openid</code> scope.

One last piece of advice. Do not write your own OAuth client or server for production — use a maintained library and a real authorization server. The value of understanding the protocol is in configuring it correctly, reading the error you actually got, and noticing when somebody proposes quietly removing one of these rules.

This guide accompanies the OAuth 2.1 + PKCE Simulator in `oauth-simulator/`. Regenerate it with `make guide` from `dev-guide/src/dev-guide.html`.